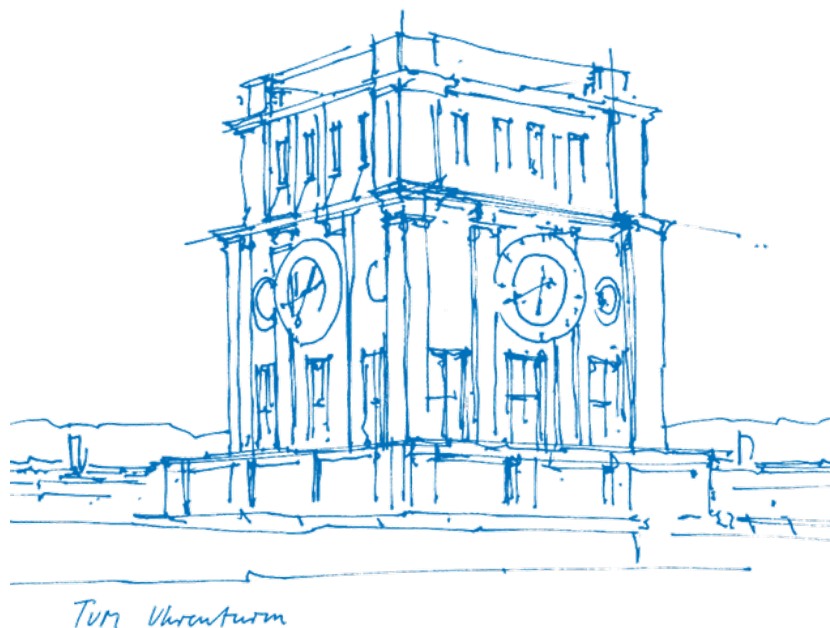


Master's Thesis in Information Systems

Xin Wen

Document-Based Process Modeler



Master's Thesis in Information Systems

Xin Wen

Document-Based Process Modeler

Dokumentatenbasierte Bearbeitung und Änderung von
Prozessmodellen

Thesis for the Attainment of the Degree
Master of Science

at the TUM School of Computation, Information and Technology,
Department of Computer Science,
Chair of Information Systems and Business Process Management (i17)

Examiner

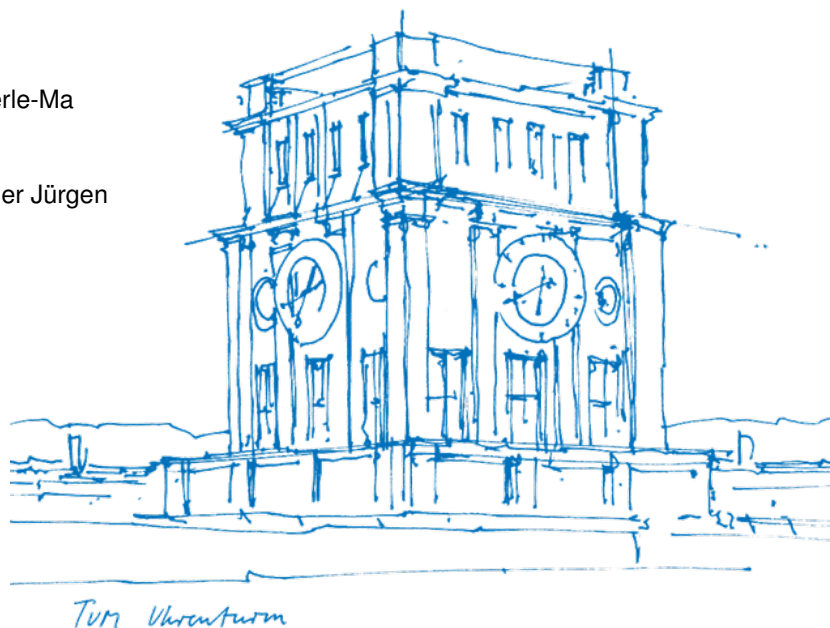
Prof. Dr. Stefanie Rinderle-Ma

Supervised by

Dr. rer. soc. oec. Mangler Jürgen

Submitted on

15.04.2026



Declaration of Academic Integrity

I confirm that this master's thesis is my own work and I have documented all sources and material used.

I am aware that the thesis in digital form can be examined for the use of unauthorized aid and in order to determine whether the thesis as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future theses submitted. Further rights of reproduction and usage, however, are not granted here.

This thesis was not previously presented to another examination board and has not been published.

Garching, 15.04.2026

Xin Wen

Abstract

Business processes are critical organizational assets; however, a large portion of process knowledge is stored in unstructured formats such as Word documents and PDF files, making the manual transformation into structured process models time-consuming and error-prone. Large language models (LLMs) have shown strong potential in supporting more efficient process modeling, but they cannot fully replace human experts, especially in domain-specific scenarios. Hybrid human–LLM process modeling is therefore a promising approach to leverage the strengths of both and compensate for their respective weaknesses. However, existing tools and approaches still mainly focus on model generation capabilities and output quality, which makes them less suitable for the scenario of creating process models directly from documents. To address this limitation, this work proposes and develops an integrated web-based tool that supports process modeling from documents through a more seamless, efficient, and user-friendly workflow. The system integrates document management, LLM-based model generation, visual traceability between text and models, subprocess relationship support, and model version control. A user evaluation with four process modeling experts shows that the system is intuitive, usable, and valuable, particularly in scenarios involving multiple models and documents.

Keywords: *Business Process Management, Process Modeling*

Contents

1	Introduction	7
1.1	Motivation	7
1.2	Research Questions	7
1.3	Contribution	8
1.4	Methodology	8
1.5	Evaluation	9
1.6	Structure	9
2	Related Work	9
2.1	Background	9
2.2	AI-Assisted Process Modeling Tools	10
3	Solution Design	13
3.1	Requirements	13
	Context and Goal	13
	Requirements Elicitation	14
3.2	User Interface and Interaction Design	16
3.3	System Architecture	17
4	Implementation	18
4.1	Frontend	18
	Architecture	18
	State Management	19
	Document Processing	20
	Text Selection Rendering	21
	Modeling	22
	Model Rendering Customization	22
4.2	Backend	23
	Architecture	23
4.3	Database Schema	24

	5
5 Evaluation	26
5.1 Evaluation Procedure	26
5.2 Evaluation Results	27
Overall Perception and Task Completion	27
Positive Aspects	28
Problems in Interaction and Use	29
Potential Improvements and Additional Features	30
Future Use Intention	31
6 Discussion	31
6.1 Usability and User Experience	31
6.2 User Expectations	32
6.3 Limitations	32
7 Conclusion	33
Bibliography	35

List of Tables

1 Literature review: horizontal research	10
2 Overview of AI-assisted process modeling tools	13
3 Summary of Evaluation Results by Functionality	29

List of Figures

1 System context	14
2 Usage model	15
3 Workspace UI Layout Mock-up	16
4 System architecture	18
5 Base <code>Store</code> class implementing the publish-subscribe mechanism	20
6 Core implementation of selection serialization	21
7 Example JSON schema used in the backend for text quotes	23
8 Data model	25

9	Modeling result for Task 1	27
10	Modeling result for Task 2	27
11	Different modeling strategies	28

Introduction

Motivation

Business processes represent core operational assets of modern organizations, defining standardized workflows, responsibilities, and execution logic for daily operations. However, a large proportion of enterprise process knowledge remains trapped in unstructured document formats, such as Word, PDF, plain text, and internal manuals. Manually converting such unstructured textual content into formal, executable process models is labor-intensive, time-consuming, and prone to human error, reducing the efficiency of process design and digitization [1].

In the era of generative artificial intelligence (AI), large language models (LLMs) have emerged as powerful enablers for automated process modeling, demonstrating strong capabilities in understanding natural language, extracting logical structures, and generating standardized process notations. Nevertheless, LLMs cannot fully replace human experts, especially in domain-specific scenarios requiring professional knowledge, contextual judgment, and structural decision-making [2]. Therefore, hybrid human-LLM process modeling has become a promising paradigm that combines human domain expertise with AI efficiency.

Despite rapid advancements, existing LLM-assisted process modeling solutions suffer from critical limitations in document-driven scenarios. Most tools adopt a “single-view, single-model” paradigm, which lacks scalability for complex systems involving multiple processes and subprocesses. To overcome these gaps, this thesis proposes an integrated tool, Document-Based Process Modeler. The tool enables a seamless, efficient, and user-centric modeling workflow with three core innovations: unified workspace management for correlated documents and models; direct text selection from uploaded documents as LLM input; and side-by-side visualization to establish explicit visual link between source text and generated process model to make a more intuitive and verifiable modeling experience.

Research Questions

This thesis addresses the following research questions:

RQ1 How do existing tools support AI-assisted process modeling from documents?

RQ2: What functionalities should a system provide to support AI-assisted process modeling from multiple documents, and how should these functionalities be designed and integrated in a seamless and user-friendly manner?

RQ3 How do user perceive the benefits and limitation of the system?

Contribution

The contribution of this thesis consists of the following aspects:

Firstly, a systematic review of the existing tools and approaches for AI-assisted process modeling is conducted, which provides insights into the current state of the art and guides the design of the proposed system.

Secondly, an integrated web-based application¹ is designed and developed to support process modeling from documents using LLMs. The system offers a more seamless, efficient, and user-friendly workflow by integrating multiple functionalities, such as document management, model generation, and traceability between text and model.

Thirdly, the proposed system is evaluated through a user study with process modeling experts. The evaluation results provide insights into the system's benefits and limitations from the users' perspective, which can inform future improvements and further research in this area.

Methodology

The research process of this thesis follows the three-cycle view of Design Science Research (DSR) proposed by Hevner [3]. This framework consists of three interconnected cycles: the relevance cycle, the design cycle, and the rigor cycle, which link the problem environment, artifact development, and the knowledge base. In this work, these cycles are applied as follows.

The relevance cycle is initiated by problems in current process modeling tools, especially the lack of support for document-based modeling. These problems are continuously discussed with modeling experts throughout the development process to identify practical needs and define system requirements.

¹<https://github.com/ga94hor/doc-based-process-modeler>, accessed 2026-04-15

The design cycle focuses on designing, implementing, and improving the artifact (RQ2), the Document-Based Process Modeler. The system is developed and evaluated iteratively. Feedback from the evaluation (RQ3) is used to refine and improve the design.

The rigor cycle is supported by continuously reviewing state-of-the-art AI-assisted process modeling tools in the literature review (RQ1), ensuring that the design is grounded in prior work and contributes to the research field.

Evaluation

The evaluation of the proposed system is done with 4 experts in qualitative way. It consisted of task-based activities and a follow-up questionnaire to collect user feedback on the tool's usability and perceived usefulness.

Structure

The remainder of this thesis is structured as follows: Chapter 2 reviews the related work in AI-assisted process modeling tools. Chapter 3 presents the design solution of the tool. Chapter 4 describes the technical implementation. Chapter 5 details the evaluation. Chapter 6 discusses the evaluation findings. Finally, Chapter 7 summarizes the thesis.

Related Work

Background

Business Process Management (BPM) refers to the systematic design, analysis, and optimization of organizational processes to improve efficiency and effectiveness, while Business Process Modeling (BPM) focuses on representing these processes in a structured and understandable form. BPM languages can be broadly categorized into conceptual, formal, and executable types. Conceptual languages, such as Business Process Model and Notation (BPMN), provide intuitive graphical notations that facilitate communication between business and technical stakeholders, although they lack fully formal semantics [4]. In contrast, formal languages like Petri Net offer mathematically precise representations, enabling rigorous analysis of properties such as correctness and deadlock-freedom [5]. To operationalize process models, executable structures such as Refined Process

Structure Tree (RPST) are used to transform models into well-structured, executable components [6].

The generation of process models from textual descriptions, commonly referred to as text-to-model approaches, has evolved significantly over time. Early work relied on rule-based methods in Natural Language Processing (NLP), mapping linguistic elements such as verbs and nouns to process activities and actors [7]. Subsequent statistical and machine learning approaches improved robustness but required annotated data and extensive feature engineering. More recently, advances in Large Language Models (LLMs) have enabled new approaches to process model generation from natural language. Recent studies indicate that LLM-based approaches have the potential to streamline modeling tasks and reduce manual effort; however, they still require careful evaluation of the generated models [1], [8].

AI-Assisted Process Modeling Tools

While many tools exist that deal with the analysis of large document sets, to the best of our knowledge, no integrated tool with the dedicated purpose of creating process models from document sets exists. Therefore, we explore related tools that support AI-assisted process modeling.

In total, eleven AI-assisted process modeling tools are identified through the literature review (see Table 1). In the following, we describe these tools with a focus on their underlying approaches and, in particular, their distinguishing functionalities.

Table 1

Literature review: horizontal research

<i>Keywords for search</i>	<i>hits</i>	<i>selected</i>	<i>Selection criteria</i>
allintitle: process modeling generative AI	12	1	Process modeling tool
allintitle: process modeling LLM[s]	16+7	5+0	Process modeling tool
allintitle: process modeling large language	49	1	Process modeling tool
allintitle: conversational process modeling	8	1	Process modeling tool
allintitle: BPMN generative AI	2	0	Process modeling tool
allintitle: BPMN LLM[s]	12+8	2+1	Process modeling tool
allintitle: BPMN large language	7	1	Process modeling tool
<i>Results horizontal search</i>	121	11	
<i>Results vertical/backward search</i>			
Added papers		1	Process modeling tool
<i>Overall:</i>		12	

Source: <https://scholar.google.com/>

In the 12 hits of the first search query ("allintitle: process modeling generative AI"), the tool ProMoAI [9] is identified. It is the first tool shown in the academic literature to leverage LLMs for automated process model generation, introduced in 2024, and is frequently referenced in subsequent work. ProMoAI enables the transformation of natural language into process models and supports iterative refinement through feedback loops, demonstrating the feasibility of LLM-based process modeling.

In the second query ("allintitle: process modeling LLM"), several tools are identified, including BPMN-Chatbot [10], BPMN Assist [11], KICoPro [12], and HyperMode [13]. BPMN-Chatbot differs from ProMoAI by focusing on token efficiency, using lightweight prompting strategies to significantly reduce computational cost while maintaining correctness. BPMN Assist distinguishes itself by introducing a structured intermediate representation (JSON) together with function-based editing, enabling incremental updates instead of full model regeneration. KICoPro takes a different direction by explicitly incorporating human-in-the-loop interaction, allowing users to guide and adjust the model generation process continuously. HyperMode further extends interaction capabilities by combining LLM-based generation with a direct manipulation interface, enabling users to operate on specific diagram elements rather than relying solely on textual prompts.

Additionally, the tool BPMNGen is firstly introduced in [14] and further developed in [15]. In contrast to purely LLM-based generation approaches, BPMNGen integrates natural language processing (NLP) techniques to analyze and structure textual process descriptions before transforming them into BPMN models. This allows the tool to better handle linguistic structures and improve the extraction of process-relevant information.

In the last search attempt combining "process modeling" with "large language", the tool PRODIGY is identified in [16]. Unlike standalone generation tools, it integrates retrieval mechanisms to incorporate external knowledge, enabling more context-aware process model construction.

In the search query ("allintitle: conversational process modeling"), the tool AutoBPMN.AI [17] is found. Its distinguishing aspect lies in directly generating executable process representations, thereby reducing the gap between process modeling and execution compared to tools that focus only on diagram generation.

Finally, combining “BPMN” with "generative AI", "LLM[s]", or "large language" leads to three additional tools: BPMN-Chatbot++ [18], LLM4BPMNGen [19], and Nala2BPMN [20]. BPMN-Chatbot++ extends the original chatbot approach by improving iterative refinement capabilities, making interaction more effective compared to earlier versions. LLM4BPMNGen differs from purely generative approaches by emphasizing more controlled and structured generation, improving output consistency. Nala2BPMN distinguishes itself through a modular pipeline architecture that separates extraction and model construction, enabling better handling of complex and unstructured input compared to end-to-end generation methods.

Overall, the reviewed tools are predominantly based on conversational, prompt-driven approaches for transforming natural language descriptions into process models. A common characteristic is the use of iterative refinement, where models are progressively improved through repeated user interaction. In terms of input, text-based descriptions represent the dominant modality, while only a limited number of tools extend this to additional forms such as voice or audio. Regarding output, the generated results are primarily expressed in BPMN. Table 2 summarizes these tools with a focus on their distinguishing characteristics.

Table 2
Overview of AI-assisted process modeling tools

Tool	Year	Input	Output	Key Contribution
ProMoAI	2024	Text	BPMN/PNML	Automated generation and iterative refinement
BPMN-Chatbot	2024	Text/voice	BPMN	Optimized for token efficiency
Nala2BPMN	2024	Text	BPMN	Hybrid LLM + algorithm pipeline
PRODIGY	2024	Text	BPMN (text)	RAG-based enterprise modeling
LLM4BPMNGen	2025	Text/voice/audio	BPMN	Audio upload + structured prompting
BPMN-Chatbot++	2025	Text/voice	BPMN	Improved prompting and interaction
AutoBPMN.AI	2025	Text	CPEE Tree	Executable model generation
HyperMode	2025	Text	BPMN	Direct manipulation + LLM interaction
BPMNGen	2026	Text	BPMN	Conversational generation + human-centered evaluation
BPMN Assist	2026	Text	BPMN	JSON-based intermediate for efficient editing
KICoPro	2026	text	BPMN	Able to answer questions and explain descisions

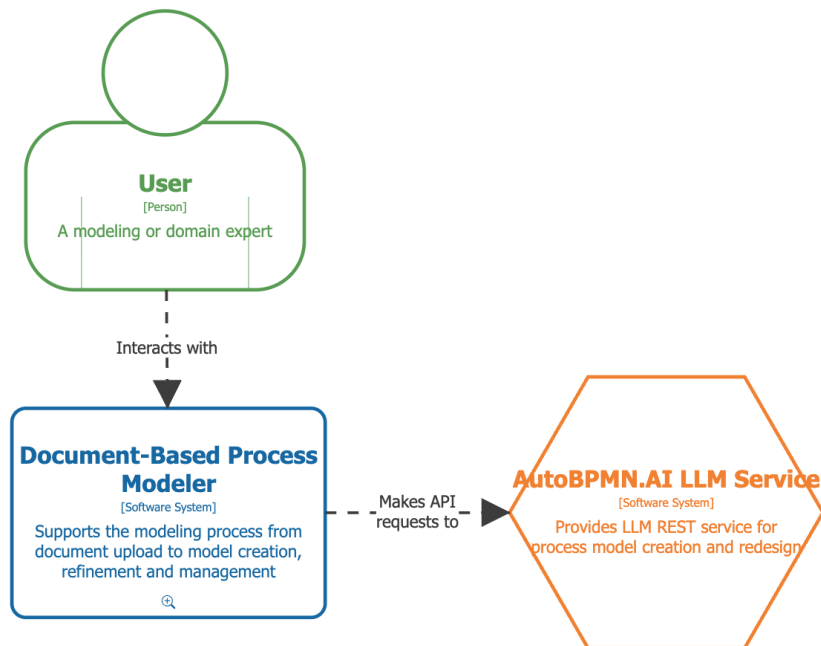
Solution Design

Requirements

Context and Goal

First, the context and objectives of the solution design are defined. The system is provided as a web-based application, as illustrated in Figure 1. The target users include both process modeling experts and domain experts who need to create process models from textual documents. To improve modeling efficiency and support non-expert users, the system leverages the LLM service of AutoBPMN.AI for automated model generation. The overall objective is to provide an integrated platform in which users can complete the entire modeling workflow—from document upload to model creation, modification, and subsequent management—in a seamless and user-friendly manner, thereby addressing RQ2.

Figure 1
System context



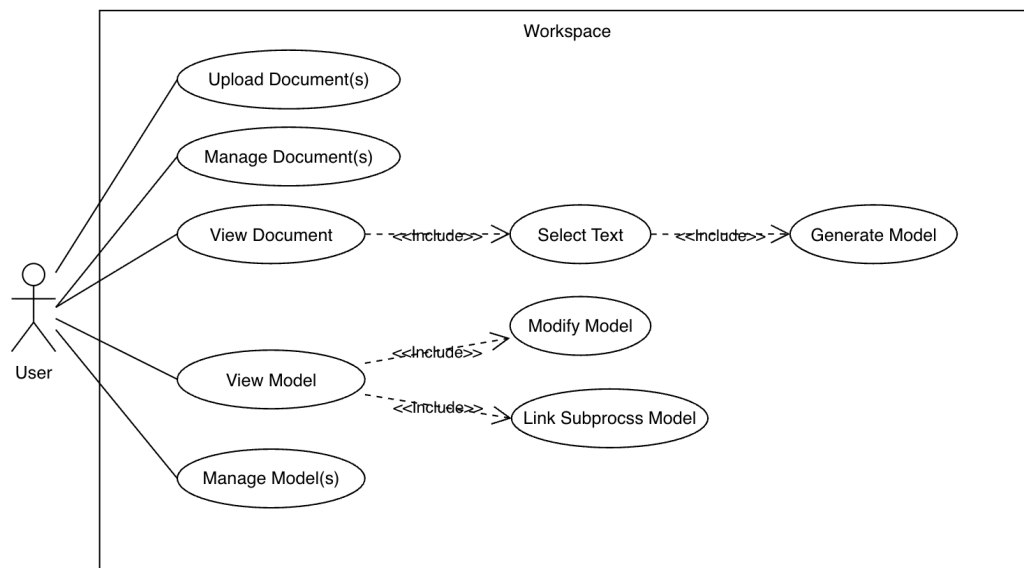
Requirements Elicitation

Functional requirements define the system's expected behavior by describing the functions, inputs, outputs, and interactions that enable the system to fulfill its intended objectives [21]. In this work, we adopt the classification proposed by [22], further diving the functional requirements into minimal requirements, which represent the indispensable core functionalities required for the system to operate, and enhancing requirements, which include additional features that improve usability and overall performance but are not strictly necessary for basic operation.

When starting the work, the following minimal requirements for the system have been elicited by my supervisor. Firstly, the system should allow users to upload multiple documents. The basic process modelling is defined as follows: first, the user should be able to select multiple passages from a single document; then, by clicking a Generate button, the system will send the selected text to the LLM service. The system should then display the generated model side-by-side with the document to facilitate verification. Besides, the system should always provide a clear visual link between the text and the generated model. Then a user should be able to select other passages to create more models. Another important requirement is that the system should allow users to establish process-subprocess relationships between models. This flow should be designed on a single web

page to avoid users having to switch between tools. The activities described are depicted in the Figure 2. Here, we use the concept of workspace for this dedicated interface, a term commonly used across many software tools, where the modeling flow is performed.

Figure 2
Usage model



Regarding non-functional requirements, only usability, as a very general goal, is defined at this stage. Besides this, a series of technical design constraints are specified. In addition to the two constraints already introduced in the system context, i.e. the system should be implemented as a web-based application and should leverage AutoBPMN.AI's LLM service², two more constraints are explicitly defined. First, the frontend should be developed using native web technologies (JavaScript, HTML, and CSS). Second, the visualization of the process model should be implemented using the CPEE Layout³ library, following the same layout as used in AutoBPMN.AI, which makes the models easy to export and import, and also provides the potential for them to be uploaded to the CPEE environment without additional manual effort, where they can be directly further manipulated and executed.

With the development of the system, several enhancing requirements were identified. One key aspect is model modification, since generated models are not final and need to be refined. The system

²<https://autobpmn.ai/>, accessed 2026-04-15

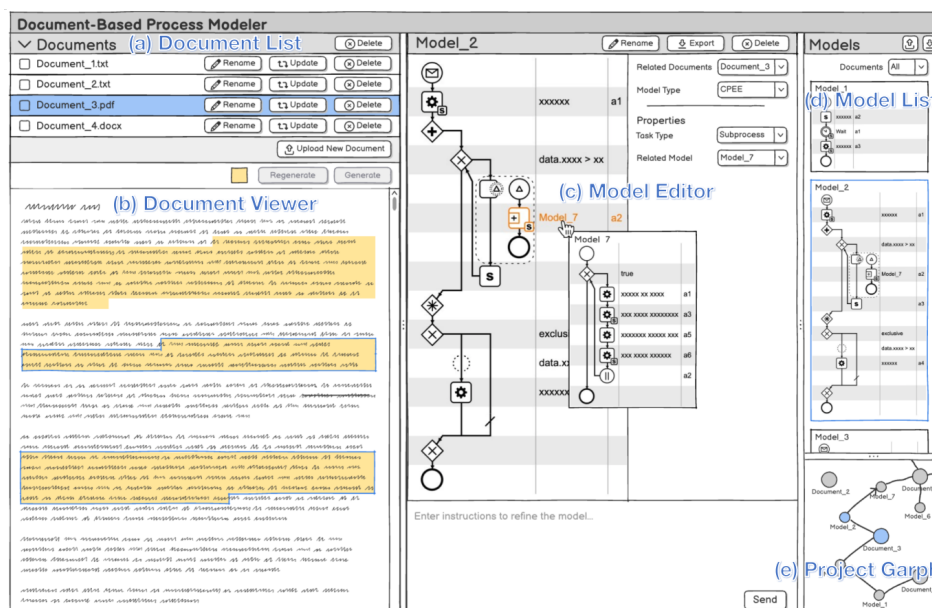
³<https://github.com/etm/cpee-layout>, accessed 2026-04-15

therefore supports several ways to modify models. Users can edit the model manually, provide additional prompts, or regenerate the model based on updated text. In addition, documents naturally require versioning, as they may be updated over time. Once document versioning is considered, model versioning becomes a natural and necessary extension in the system. Since models are created based on documents, they may need to follow these changes. Supporting model versioning helps keep the relation between documents and models clear and manageable.

User Interface and Interaction Design

The layout design of the workspace page is illustrated in Figure 3 as a three-pane layout. The left pane is dedicated to document uploading and viewing, the middle pane supports process modeling, and the right pane is used for process model visualization and project graph representation. The components of the interface are organized as follows: (a) Document List, (b) Document Viewer, (c) Model Editor, (d) Model List, and (e) Project Graph. The Document List allows users to manage their uploaded documents, while the Document Viewer provides a space for users to read and select text from the documents. The Model Editor is where users can view, edit and refine the process model. The Model List displays all the created models for easy access. The Project Graph visually represents the relationships among models and documents, particularly the process-subprocess model links.

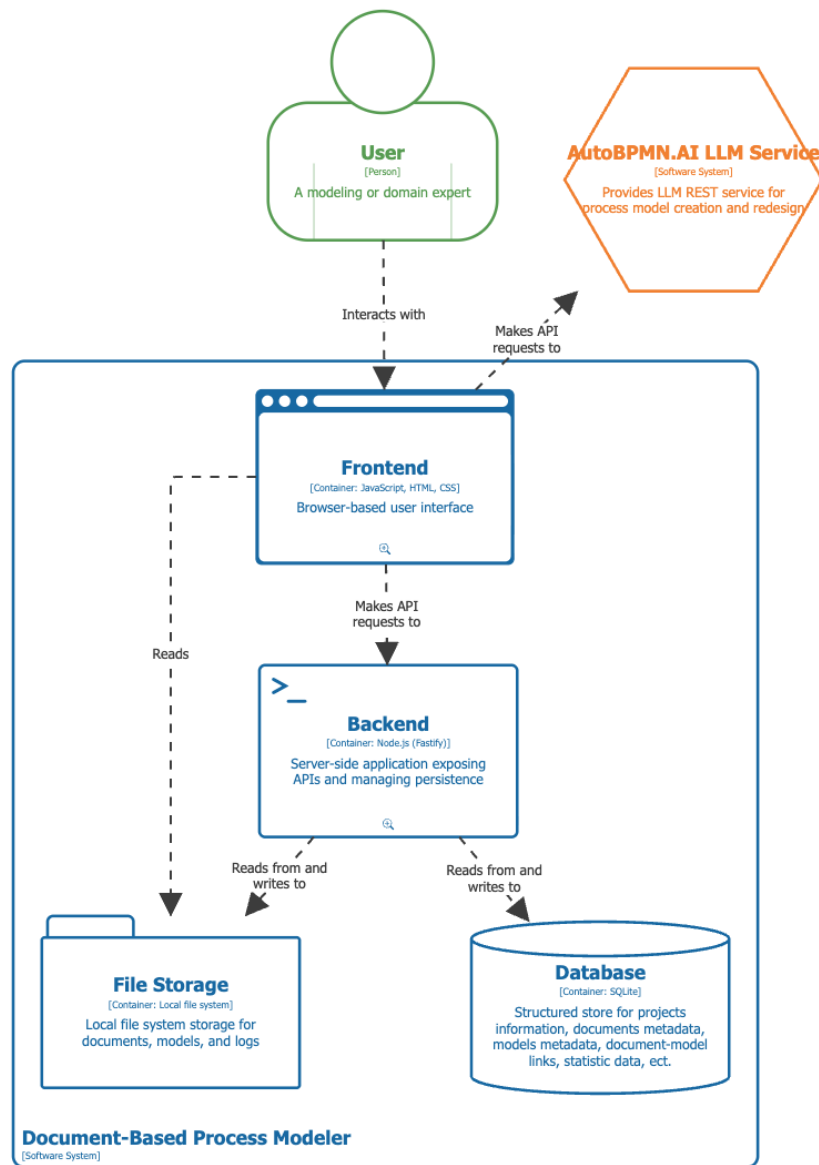
Figure 3
Workspace UI Layout Mock-up



System Architecture

The system architecture is illustrated in Figure 4. It follows a client–server architecture consisting of a frontend, a backend, and a persistence layer, with integration of an external LLM service. The frontend is implemented as a multi-page application (MPA), consisting of a home page, a workspace page, and a statistics page, where the workspace page represents the core of the system, supporting document handling and process model generation. For LLM-based model generation, the frontend sends requests to the external LLM service directly, and the generated results are returned to the frontend for user review and interaction. After user confirmation, the data is transmitted to the backend for persistence. The frontend communicates with the backend via RESTful APIs, through which the backend manages projects, documents, and models and handles data storage. The persistence layer follows a hybrid approach: the actual content of documents and models is stored in the local file system, while their metadata (e.g., version numbers and names) and application data, such as project information and relationships between process models, are maintained in a database.

Figure 4
System architecture



Implementation

Frontend

Architecture

The frontend was implemented as a native web application using JavaScript, HTML, and CSS. Since the main modelling workflow is performed in the workspace page, only the workspace architecture

is described here. The workspace follows a layered structure: UI modules handle rendering and user interaction, service modules coordinate the workflow logic, and store modules keep the client-side state synchronized across the document viewer, model editor, and project graph.

The workspace initialization logic validates the current project, loads the required UI modules, and triggers the initial data loading. At runtime, the workspace service loads document, model, and subprocess metadata, initializes the stores, and selects an initial document if available. Later actions, such as opening a version, generating a model, or navigating to a subprocess, are handled through the same orchestration flow.

State Management

The workspace presents four concurrently visible panes — the document list, document viewer, model editor, and project graph — each of which must react to changes triggered in the others. To keep these panes synchronized without coupling them directly, the frontend adopts a lightweight publish-subscribe mechanism: each pane subscribes to the stores it depends on and is notified whenever relevant state changes.

The mechanism is built around a small store hierarchy. The base `Store` class, shown in Figure 5, holds local state in a plain object and maintains a subscriber set. Specialized stores extend this foundation to cover different concerns: `EntitiesStore` manages keyed entity collections and exposes add, update, and remove operations; `VersionedEntityStore` further extends it with version-aware lookups; and separate stores for workspace coordination, document-viewer state, model-editor state, and the project graph each encapsulate their own domain fields and mutations.

Figure 5*Base Store class implementing the publish-subscribe mechanism*

```

export class Store {
  constructor(initialState = {}) {
    this.state = { ...initialState };
    this.subscribers = new Set();
  }

  subscribe(fn) {
    this.subscribers.add(fn);
    return () => this.subscribers.delete(fn);
  }

  notify(patch) {
    this.subscribers.forEach((fn) => fn(patch));
  }
}

```

UI modules interact with stores by calling `subscribe()` during initialization and holding the returned unsubscribe function for cleanup. Each notification delivers a patch object carrying a `key` and an `operation` field, so subscribers can react selectively — for example, the model-editor panel handles model-editor store patches while ignoring document-viewer updates. This targeted notification pattern supports synchronized updates across the workspace panes without requiring a larger frontend framework.

Document Processing

Document upload is partly handled on the frontend so that heterogeneous source formats can be normalized before entering the main interaction flow. The implementation accepts PDF, DOC, DOCX, and plain-text input. Word documents are converted to HTML using Mammoth.js⁴, a client-side library that maps DOCX content to semantic HTML elements. PDF files are processed

⁴<https://github.com/mwilliamson/mammoth.js>, accessed 2026-04-15

using PDF.js⁵, Mozilla’s open-source PDF rendering library, which provides access to the raw text content and its positional metadata.

For PDFs, the frontend uses PDF.js to extract per-page text items, then groups them into lines based on their vertical positions, classifies them as headings, list items, or paragraphs using font size and formatting heuristics, and assembles the result as semantic HTML. This normalization step is necessary because later operations — selection serialization, overlay rendering, and re-anchoring — depend on stable DOM text ranges rather than on raw file content.

Text Selection Rendering

One of the central frontend mechanisms is the transformation of browser text selections into persistent, version-aware linkable objects. A user selection is serialized into a text-position selector together with a text-quote selector, following the anchoring idea of the W3C Web Annotation Model [23]. The text-position selector identifies the range in the current text stream, while the text quote preserves the selected wording for later validation or re-anchoring. Figure 6 shows the core implementation used to convert a browser range into a persistent selector object.

Figure 6

Core implementation of selection serialization

```
export function serializeRange(range) {
  const root = getRoot();
  const textPosition = textPos.fromRange(root, range);
  const textQuoteSelector = textQuote.fromTextPosition(root, textPosition
    );

  return {
    textPosition,
    textQuote: textQuoteSelector,
  };
}
```

⁵<https://mozilla.github.io/pdf.js/>, accessed 2026-04-15

When a stored selection is displayed again, the frontend resolves the serialized selector back into a DOM range. This allows existing links and older model versions to be visualized in the document viewer.

Virtual Overlay Strategy. Selection rendering is implemented as a virtual overlay rather than by wrapping the original text nodes. The selection module collects client rectangles, filters and de-duplicates them, removes oversized container rectangles, and reconstructs more accurate line-level rectangles when necessary. The resulting geometry is then rendered as overlay elements above the document content. This avoids mutating the source DOM and keeps selection rendering stable across re-renders, updates, and version switches.

Link Update Detection. The frontend also distinguishes between semantic changes in a linked selection and purely visual updates. Link drafts are classified into three states: no change, color-only change, and text-changed. This distinction is important during document-version updates, where a visual modification should not invalidate the semantic relation between a passage and a model.

Modeling

The modeling flow coordinates the document viewer, the model service, and the model editor state. Selected text is aggregated, optionally combined with an additional prompt, and sent to the LLM-backed generation endpoint. The returned XML model data is then loaded into the model editor store and rendered in the editing pane.

The frontend also keeps the relation to the originating document version so that model editing remains synchronized with the document context. This logic supports new models, regenerations, refinements, and historical versions.

Model Rendering Customization

Instead of implementing a process-model editor from scratch, the frontend reuses the CPEE layout and rendering infrastructure and extends it where necessary. The main customization is the workflow adaptor layer, which dispatches additional events for call and subprocess nodes.

These events connect the generic renderer to workspace-specific behavior such as subprocess navigation and contextual interaction. Together, the layered frontend organization, pub-sub stores, document normalization, overlay rendering, and renderer customization support the end-to-end modelling workflow.

Backend

Architecture

The backend is implemented using a layered architecture. In this system, the layered structure is realized through route, service, and repository layers, each with a clearly separated responsibility. Route modules expose REST endpoints, service modules implement the application logic, and repository modules encapsulate data-access operations. Fastify.js was chosen as the backend framework because it is lightweight and provides native support for schema-based validation of requests and responses.

The schema layer is important because the system exchanges structured selection data between frontend and backend. As shown in Figure 7, selectors are validated before entering the service logic. The same approach is used across document, model, and link endpoints.

Figure 7

Example JSON schema used in the backend for text quotes

```
const textQuoteSchema = {
  type: "object",
  required: ["exact"],
  additionalProperties: false,
  properties: {
    exact: { type: "string" },
    prefix: { type: "string" },
    suffix: { type: "string" },
  },
};
```

The backend combines relational persistence with file-based storage. Metadata such as projects, documents, models, versions, and links are stored in SQLite, while large document contents and

model XML payloads are stored as files in the persistence directory. This hybrid design keeps metadata queryable without placing large HTML and XML payloads directly in the database.

The service layer implements the application-specific workflows used by the frontend. For example, when a new document version is created, the backend stores the content, updates the latest-version pointer, and copies the latest link information. When a new model or model version is created, the backend persists the XML payload, updates version references, stores the related link, and records monitoring information. The backend also handles cross-cutting concerns such as soft deletion, logging, and aggregated workspace endpoints.

Database Schema

Figure 8 illustrates the current data model of the system. Firstly, Project, Document, Model and DocumentModelLink are the minimal tables needed to support the proposed system. It is also how the system was implemented when it first adopted the database. The Document and Model tables were used to store metadata like name, words count of a document. Theoretically, the DocumentModelLink table can be stored as an attribute within the ModelVersion, since the system is currently designed to allow one model to be derived from only one document. But it is a decision made very early to store it in a separate table to easily support a potentially many-to-many (M2M) relationship in the future.

The ModelVersion table is added when the model versioning feature is about to be implemented. The DocumentVersion table is added at the same time to prepare for supporting the document update feature, as both tables follow a similar logic at the DB operation level.

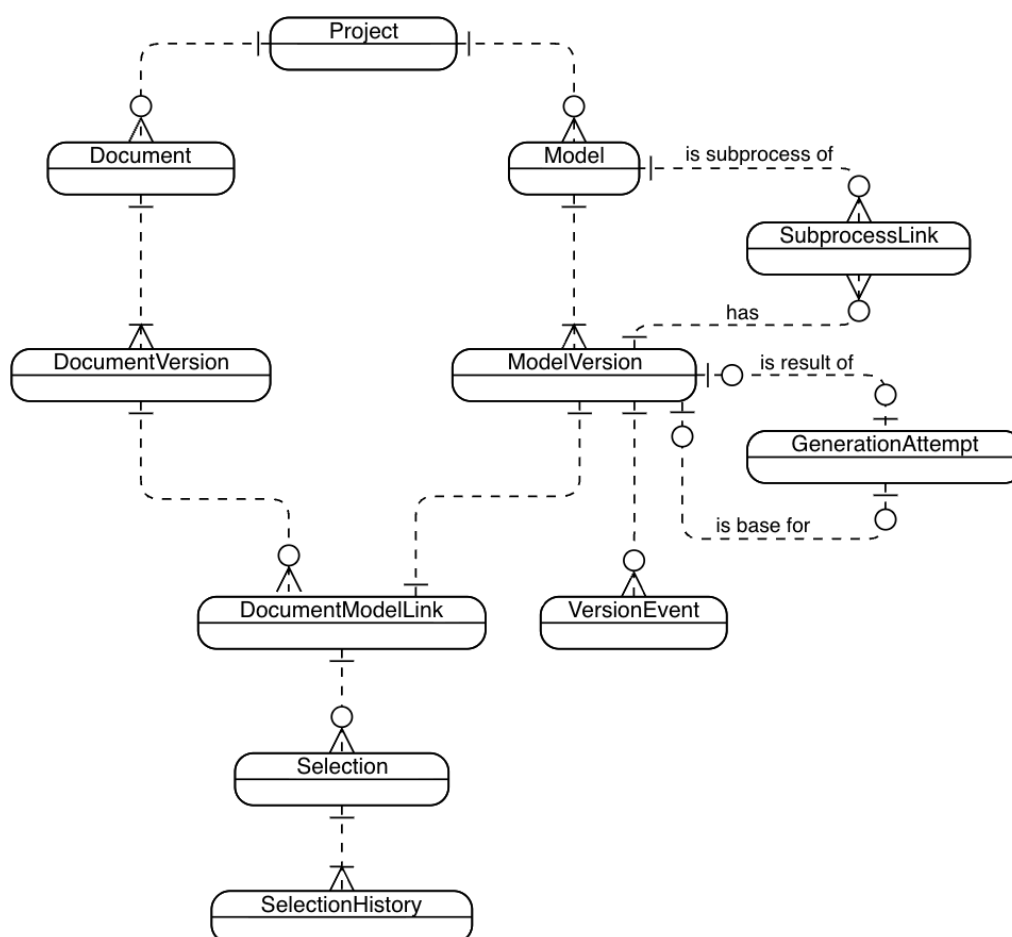
The SubprocessLink table is an additional table used to support the project graph. As this relationship information has been embedded in model data, the model visualization itself does not need this data. However, for the project graph, the relationship is hard to be parsed from the model data; therefore, such a table is implemented to easily retrieve the process-subprocess relationship.

The Selection and SelectionHistory tables are then added when designing the Document Update feature. Before the selection data (e.g., color, position) for a DocumentModelLink is stored as an attribute in the format of a stringified JSON object. To better support auto-reanchoring for document update, Selection and SelectionHistory are implemented as separate tables. The separate Selection

table can make selection data easier to retrieve for reanchor operations. Besides, a Selection History is designed to make it easier to track the quality of reanchor.

Finally, the GenerationAttempt and VersionEvent tables are the two tables needed to monitor the system behavior. The GenerationAttempt table is used to monitor the accept/decline decision for the LLM-generated output. The VersionEvent is mainly to store the manual edit behavior or a system auto-update behavior, e.g., selection reanchor for document update.

Figure 8
Data model



Physical data model of the system with attributes omitted

Evaluation

Evaluation Procedure

The developed system was evaluated with four modeling experts to assess its usability and perceived usefulness, as well as to gather feedback on potential improvements and additional features. The evaluation process included a short demonstration, task-based activities, and a questionnaire. Each evaluation session lasted approximately 40 minutes.

The session was structured as follows: It began with a demonstration to introduce the tool's end-to-end workflow and key features. After that, the participants were asked to complete two tasks. The first was an exploratory task based on a fictional B2B order-processing scenario, consisting of two wiki-style documents⁶ in DOCX format. It was designed to help participants familiarize themselves with the system. The second was a more realistic process modeling task based on an open-source SOP document⁷ in PDF format. For both tasks, participants were not given strict instructions on how to proceed, allowing them to explore the system freely. Due to the known issue of the PDF file rendering, the link to the online version was provided. If time permitted, participants could additionally use the system with their own documents.

Following task completion, participants completed a questionnaire including the following open-ended questions:

- (1) What aspects of the tool did you like while using it?
- (2) What aspects of the tool did you not like or find problematic?
- (3) What improvements or additional features would you most like to see in the tool?
- (4) If you needed to model processes from documents in the future, would you consider using this tool? Why or why not?

⁶https://github.com/ga94hor/doc-based-process-modeler/tree/evaluation_260324/evaluation_data/Task1_Fictional_B2BOrderProcessing, accessed 2026-04-15

⁷https://dnrc.mt.gov/_docs/forestry/Wildfire/agreements-plans-guides/DNRC-Manuals/300-Fire-Business-Manual/Appendix-300-Manual/2023_Fire_Contract_PMT_SOP.pdf, accessed 2026-04-15

Evaluation Results

Overall Perception and Task Completion

Overall, participants perceived the tool as usable and intuitive. Figure 9 and Figure 10 show the modeling results from one expert for the first and second tasks, respectively.

Figure 9
Modeling result for Task 1

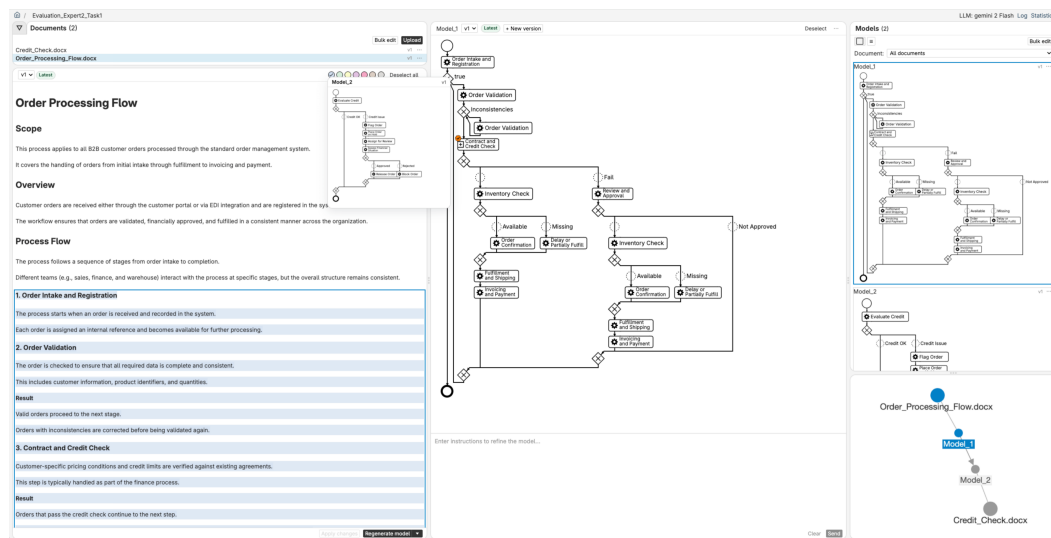
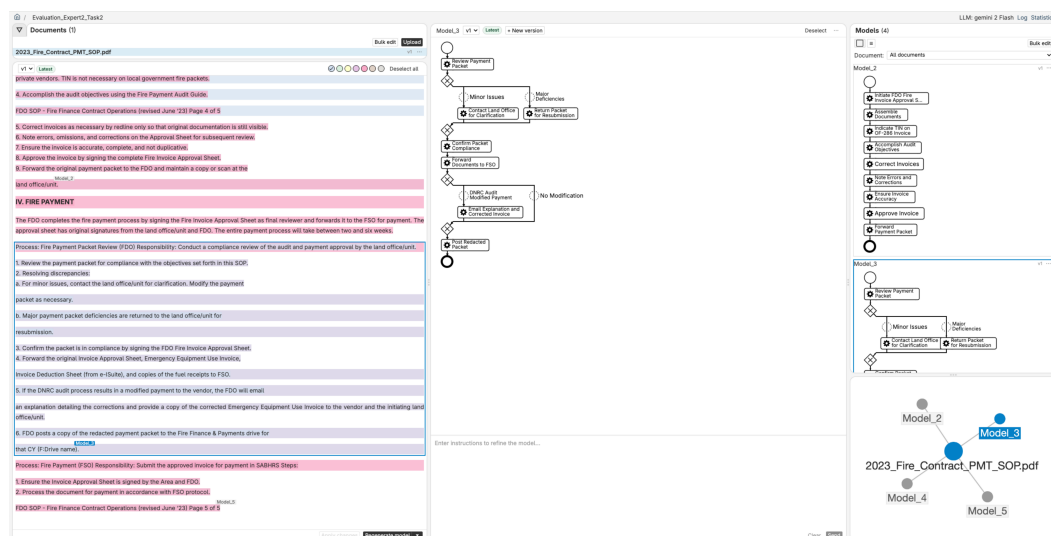
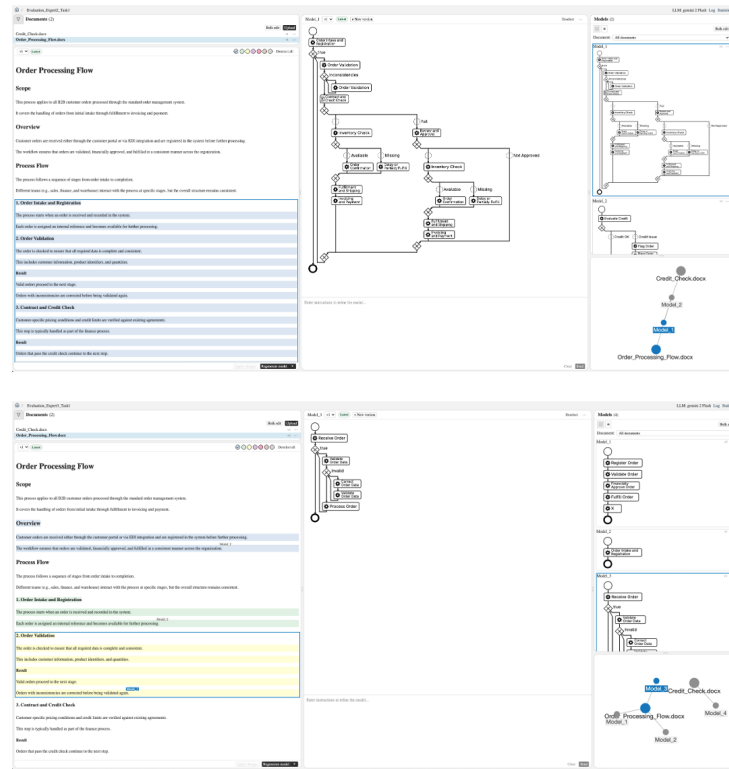


Figure 10
Modeling result for Task 2



One interesting observation is that while most participants tended to start with, or at least try once, selecting larger portions or all of the text when reading a new document, one participant began by creating models from smaller segments. These different strategies are illustrated in Figure 11.

Figure 11
Different modeling strategies



Different modeling strategies observed during task execution (above: larger part selection; below: step-by-step selection)

For detailed task completion and feedback, the participants showed varied interests and offered differing perspectives on the system. The result on each feature is summarized in Table 3, and will be described in detail in the following sections.

Positive Aspects

Firstly, the following aspects are generally perceived positively by the participants: the performance of the integrated LLM service, the easy-to-establish process-subprocess relationship, and model version control.

Furthermore, several additional positive aspects were mentioned. First, the embedding of relevant information, such as selected text and prompt history, within the model's XML data is considered a

Table 3
Summary of Evaluation Results by Functionality

Functionality	Positive Aspects		Problems and Limitations	
	Item	n	Item	n
Document Rendering and Interaction			Readability issue	1
			Expected AI-assisted search capability	1
Modeling Process			Unintuitive to start a new modeling by deselecting the current model	2
			Unnecessary review step (of the initial generated model)	2
Integrated LLM Service	Fast	2	Lack of transparency in the conversational modeling logic	2
	Good quality	2		
Model Versioning	Helpful	3	Expected version comparison	1
Augmented Model Data	Well-considered	2		
Text-Model Link	Clear	1	Expected fine-grained traceability	2
Process-Subprocess Relationship	Convenient	3		
Project Graph	Helpful for larger systems	1	Unclear node types and version representations	2
			Expected more expressive visualization	1
Log and Statistics	Potentially useful for analysis	1		

n = number of participants who mentioned each item

well-integrated design feature. Secondly, logging and statistical information elicited interest among some participants due to its potential for further analysis. Thirdly, the project graph visualization was mentioned as helpful, especially for larger systems.

Problems in Interaction and Use

The following outlines the main problems and challenges reported by participants, as well as notices observed during the evaluation.

First, for the document’s rendering and interaction, only one feedback regarding the readability issue of PDF documents was mentioned in feedback. But it is observed that one participant started with reading the original PDF online firstly and one participant tried to use browser-extension to search for keywords.

Regarding the modeling process design, two problems were mentioned. First, the step to start a new modeling by a explicit deselection step of the current model was mentioned. Two reply received in the feedback, but it is a feature generally perceived as not intuitive immediately and needs additional clarification. Second, due to the sufficient good quality of the initial model, the review step was mentioned by one participant as totally unnecessary as there is version control and by another one as the initial generated model was mentioned as an unnecessary step.

Another main problem mentioned and observed is that the system’s conventional modeling logic is not transparent to users and needs additional explanation. While the initial model generation from text was relatively clear, the model modification process was less transparent. They did not understand how their inputs were processed or how the system used the selected text and existing model.

Potential Improvements and Additional Features

Besides the problems found during system use, many valuable insights were collected.

For the existing features, improvements to the project graph visualization were mentioned most often. This includes clearer differentiation and identification of elements (i.e., what type of element a node is and which version it represents), as well as using more expressive visual designs, such as size, color saturation, and distance, to show additional information beyond type.

For document-related features, an AI-assisted document search and interaction function was suggested. This would allow users to query the document by providing an instruction, after which the retrieved parts can be automatically selected and used directly or as a basis for further adjustment for model generation.

For the process modeling features, one notable desired improvement emphasized by the participants is the need for finer-grained traceability between source text and model. When selecting larger text

passages, only a subpart of the text is actually used and reflected in the generated model, and it is unclear which parts are actually used. Therefore, a clearer indication of the actual text used for model generation is required, ideally at the level of individual tasks. Secondly, model comparison was suggested as a valuable feature to help users understand and verify changes between different model versions, making it easier to identify modifications and track model iterations over time. Beyond this, participants also expected more general advanced AI modeling capabilities, including explainability of the generated model through conversational interaction with chatbots, as well as support for model quality evaluation and automatic improvement.

Future Use Intention

Three participants indicated an intention to use the system in scenarios involving the creation of multiple models. In such cases, the system's features, such as text-model link, process-subprocess relationship, and model management functionalities, were considered helpful. One participant additionally raised the concern that automatically generated models may bypass the user's need to fully understand the underlying document, potentially introducing bias. Hence, the fine-grained traceability, as suggested by the participant, could help mitigate this problem. Another participant again pointed out the need for the AI-assisted document search capability as suggested.

Discussion

This chapter discusses the evaluation's findings and reflects on their implications for system design.

Usability and User Experience

Among these usability issues, some can be mitigated and improved through design, while others are more challenging to be improved for the system.

The first finding is that document readability is key to the tool's design, as it aims to enable users to start using it by reading. If a user doesn't have a good reading experience, turns to reading the document in their familiar way, and then finds the same text position within the tool, it will significantly affect the tool's designed usage scenario and reduce efficiency.

For the interaction design, some of the improvement ideas have been made up; e.g., to avoid the unnecessary review step, we can easily offer a project settings feature. And for the transparent AI input or system logic, it can be improved by showing instructions, e.g., using a switch button to give the user more flexible control over the AI input. In contrast, as this tool introduces a new way of process modelling involving multiple documents and multiple models, e.g., deselecting the current model to start a new model step, is quite representative, without coming up with a good solution, therefore, it requires users to be familiar with the new interaction and workflow, which raises the challenge for the system.

User Expectations

During the evaluation, participants raised more advanced expectations, which are out of the current system's scope, but are worth further discussion.

Firstly, the expected fine-grained traceability between text and model represents an advanced requirement that goes beyond the capabilities of current LLM-based process modelling tools. However, it is a particularly important feature in the context of document-based process modelling, as the system is designed to facilitate a easier verification for the traceability of the AI-generated model. Therefore, exploring approaches to enable fine-grained traceability can be considered a valuable direction for future work.

Secondly, the expectation of integrated AI-powered document intelligence is another advanced requirement that has not yet been included in the current system. However, it represents a promising and valuable direction for future development, as such capabilities could further enhance the system's efficiency and perceived usefulness. In particular, enabling intelligent document understanding, contextual retrieval, and semantic linking between textual content and modelling elements could significantly streamline the modelling workflow and improve the overall user experience.

Limitations

Due to the small sample size ($n=4$), all of whom are modelling experts, the study did not include the perspective of domain experts. Besides, although the material provided is a real-world SOP document, it is a really well-written one with appropriate structure and not too long, 5 pages. So the document readability issue could affect the tool's usability more.

Besides, the evaluation didn't mention it as a limitation that the system is currently designed so that a model can be generated from a single document, without enabling cross-document modeling, which may require a more complex user interaction design.

Conclusion

This work is motivated by the inconvenience of state-of-the-art AI-assisted process modeling tools, which typically only provide a text input box and lack of support for document-based sources. To resolve this issue, we designed, implemented, and evaluated an integrated web-based process modeling tool⁸, especially for the document-based process modeling scenario, offering a seamless, efficient, and user-friendly workflow.

We first revisit the research questions:

RQ1 How do existing tools support AI-assisted process modeling from documents?

A thorough systematic review of existing tools and approaches for AI-assisted process modelling is conducted. The review shows that most of the existing tools still focus on model generation capabilities and quality. There is a trend to augment the input model with audio. However, the use case of creating process models from documents or a segment of documents remains missing.

RQ2: What functionalities should a system provide to support AI-assisted process modelling from documents, and how should these functionalities be designed and integrated in a seamless and user-friendly manner?

We derived minimal core requirements as follows: a unified workspace for documents and models; direct text selection from documents as LLM input; side-by-side visualization with explicit text-model traceability; and subprocess relationship management. Additional enhanced features include LLM-based model refinement, model versioning, document update, bulk management, and interactive project graph visualization.

RQ3 How do users perceive the benefits and limitations of the system?

⁸<https://github.com/ga94hor/doc-based-process-modeler>, accessed 2026-04-15

An evaluation with four process modeling experts indicates that the system is generally intuitive and usable, with particular value for creating and exploring multiple models from documents, supported by features such as text–model links and process–subprocess links. The evaluation implies two valuable future work directions: fine-grained text–model traceability and AI-powered document intelligence, both of which have particular potential to improve efficiency and user experience.

Overall, this work demonstrates the feasibility and practical value of supporting document-based process modeling through the developed prototype. It introduces a novel input modality that enables direct text selection from uploaded documents as LLM input. Future work will focus on further improving the system’s usability, extending support to additional document types, and conducting research in fine-grained traceability and the integration of AI-powered document intelligence.

Bibliography

- [1] N. Daclin, S. Mallek-Daclin, and G. Zacharewicz, “Generative ai for business model generation (gai4bm): From textual description to business process model,” in *10th International Food Operations and Processing Simulation Workshop (FoodOPS)*, 2024. DOI: 10.46354/i3m.2024.foodops.013.
- [2] N. Klietsova, J.-V. Benzin, J. Mangler, T. Kampik, and S. Rinderle-Ma, “Process modeler vs. chatbot: Is generative ai taking over process modeling?” In *International Conference on Process Mining*, Springer, 2024, pp. 637–649.
- [3] A. R. Hevner, “A three cycle view of design science research,” *Scandinavian Journal of Information Systems*, vol. 19, no. 2, pp. 87–92, 2007.
- [4] Object Management Group, *Business process model and notation (bpmn) version 2.0*, <https://www.omg.org/spec/BPMN/2.0>, 2011.
- [5] W. M. P. van der Aalst, “The application of petri nets to workflow management,” *Journal of Circuits, Systems, and Computers*, vol. 8, no. 1, pp. 21–66, 1998.
- [6] J. Vanhatalo, H. Völzer, and J. Koehler, “The refined process structure tree,” in *Proceedings of the 7th International Conference on Business Process Management (BPM)*, Springer, 2009, pp. 100–115.
- [7] F. Friedrich, J. Mendling, and F. Puhlmann, “Process model generation from natural language text,” in *Proceedings of the 23rd International Conference on Advanced Information Systems Engineering (CAiSE)*, Springer, 2011, pp. 482–496.
- [8] H. Kourani, A. Berti, D. Schuster, and W. M. P. van der Aalst, “Process modeling with large language models,” *arXiv preprint arXiv:2403.07541*, 2024.
- [9] H. Kourani, A. Berti, D. Schuster, and W. M. Van Der Aalst, “Promoai: Process modeling with generative ai,” *arXiv preprint arXiv:2403.04327*, 2024.
- [10] J. Köpke and A. Safan, “Introducing the bpmn-chatbot for efficient llm-based process modeling,” in *BPM (Demos/Resources Forum)*, 2024, pp. 86–90.
- [11] J. T. Licardo, N. Tanković, and D. Etinger, “Bpmn assistant: An llm-based approach to business process modeling,” *Applied Sciences*, vol. 16, no. 5, p. 2213, 2026.

- [12] C. Lauer, P. Pfeiffer, and N. Mehdiyev, “Human-centered evaluation of an llm-based process modeling copilot: A mixed-methods study with domain experts,” *arXiv preprint arXiv:2603.12895*, 2026.
- [13] S. Pardo Gutierrez, L. Alrabie, G. Di Fede, M. Vitali, and S. Andolina, “A direct manipulation interface for llm-based process modeling,” in *Proceedings of the 16th Biannual Conference of the Italian SIGCHI Chapter*, 2025, pp. 1–3.
- [14] L. F. Hörner, “Towards an llm-based conversational framework for business process modeling: Research approach and preliminary results,” in *International Conference on Advanced Information Systems Engineering*, Springer, 2025, pp. 286–293.
- [15] L. F. Hörner, M. Möller, and M. Reichert, “Automatically generating bpmn 2.0 process models from natural language process descriptions: Challenges, framework, quality assessment,” *Business & Information Systems Engineering*, pp. 1–25, 2026.
- [16] C. Ziche and G. Apruzzese, “Llm4pm: A case study on using large language models for process modeling in enterprise organizations,” in *International conference on business process management*, Springer, 2024, pp. 472–483.
- [17] N. Klievtsova, M. Ehrendorfer, J. Mangler, and S. Rinderle-Ma, “Autobpmn. ai: Conversational process modeling and automation,” *CEUR Workshop Proceedings*, vol. 4032, pp. 304–311, 2025, issn: 1613-0073.
- [18] A. Safan and J. Köpke, “Bpmn-chatbot++: Llm-based modeling of collaboration diagrams with data,” in *Proceedings of the BPM*, 2025.
- [19] A. Costa, A. Wimmer, and L. Pufahl, “Llm4bpmngen: A tool for bpmn generation with llms,” 2025.
- [20] A. Nour Eldin, N. Assy, O. Anesini, B. Dalmas, and W. Gaaloul, “Nala2bpmn: Automating bpmn model generation with large language models,” in *International conference on cooperative information systems*, Springer, 2024, pp. 398–404.
- [21] Wikipedia contributors, *Functional requirement*, [Online; accessed 14-April-2026], 2026. [Online]. Available: https://en.wikipedia.org/wiki/Functional_requirement.
- [22] D. Randall Brandt, “How service marketers can identify value-enhancing service elements,” *Journal of Services Marketing*, vol. 2, no. 3, pp. 35–41, 1988.

- [23] W3C, “Web annotation data model,” World Wide Web Consortium, Tech. Rep., 2017, Accessed: 2026-04-05. [Online]. Available: <https://www.w3.org/TR/annotation-model/>.